TECHNICAL REPORT / CPU BACKENDS

Compiling scheduled egglog rules into reusable CPU execution blocks

An architecture proposal for egglog and eggcc, evaluated on every workload in the passing suite at eggcc revision `ae80f82`.

Eggfuse compiles rule-local joins and effects into typed recipes, shares entry work across consumers, and carries checked references to committed constructor results across scheduled Runs. It leaves Free Join responsible for residual matches. The architectural proposal is to expose these operations as optional physical plans inside egglog, where the native engine can retain ownership of scheduling, merges and extraction.

Frozen inference	Tasks	Cell faster	Geomean N/C	Total N/C
Pass 1	96	93	1.247×	1.176×
Pass 2	96	62	1.113×	1.031×
Pass 3	96	64	1.077×	1.260×
Three-pass sum	96	73	1.138×	1.131×

N/C is original unmarked egglog replay time divided by Eggfuse replay time. Ratios above 1 favor Eggfuse. Each row uses all paired successes in that scope. Appendix A gives every task. Appendix B gives the original eggcc production timings.

Across 96 tasks, the three captured schedules took 143.426 s in native egglog and 126.802 s in Cell in aggregate. Preparation of the Cell targets took an additional 791.6 s in this experiment. Raytrace contributed 82.6% of the aggregate time saved. Excluding it gives a time-weighted ratio of 1.029×. A fixed sum-bits block catalog was transferred to every target without per-target block training. The survey measures isolated, single-thread inference. Each next pass starts from eggcc's captured production input. A separately repeated comparison appears in §6.

> **The adoption decision.** Prototype shared entry packets and checked register joins beneath egglog's existing evaluator. The current standalone backend has a concrete sum-bits endpoint discrepancy even though projected table counts agree. Treat the timing differences as architecture evidence. Require the native observable-result and extraction gates in §7 before enabling an alternative backend for production.

Revision of 6 November. Section 9 adds a separate two-workload memory study. The full-suite measurements below use their original snapshot. Source snapshot `70b2910` · Apple M3 Max · arm64 macOS 27.0.1 · Rust 1.94.0. The corpus has 96 Bril/Rust workloads and 288 pass inputs. Scope, individual timings, replay scripts, witness and input digests are included in the companion artifact.

1. Where the compiler meets the engine

Eggcc supplies a scheduled equality-saturation program and uses an extractor to choose a result for its next pass. Egglog supplies incremental relational evaluation and congruence closure. A useful extension point is the compiled rule: it already identifies variables, table calls, key columns, effect destinations and ruleset ownership. Eggfuse makes those relationships explicit for a standalone executor. eggcc source · egglog semantics

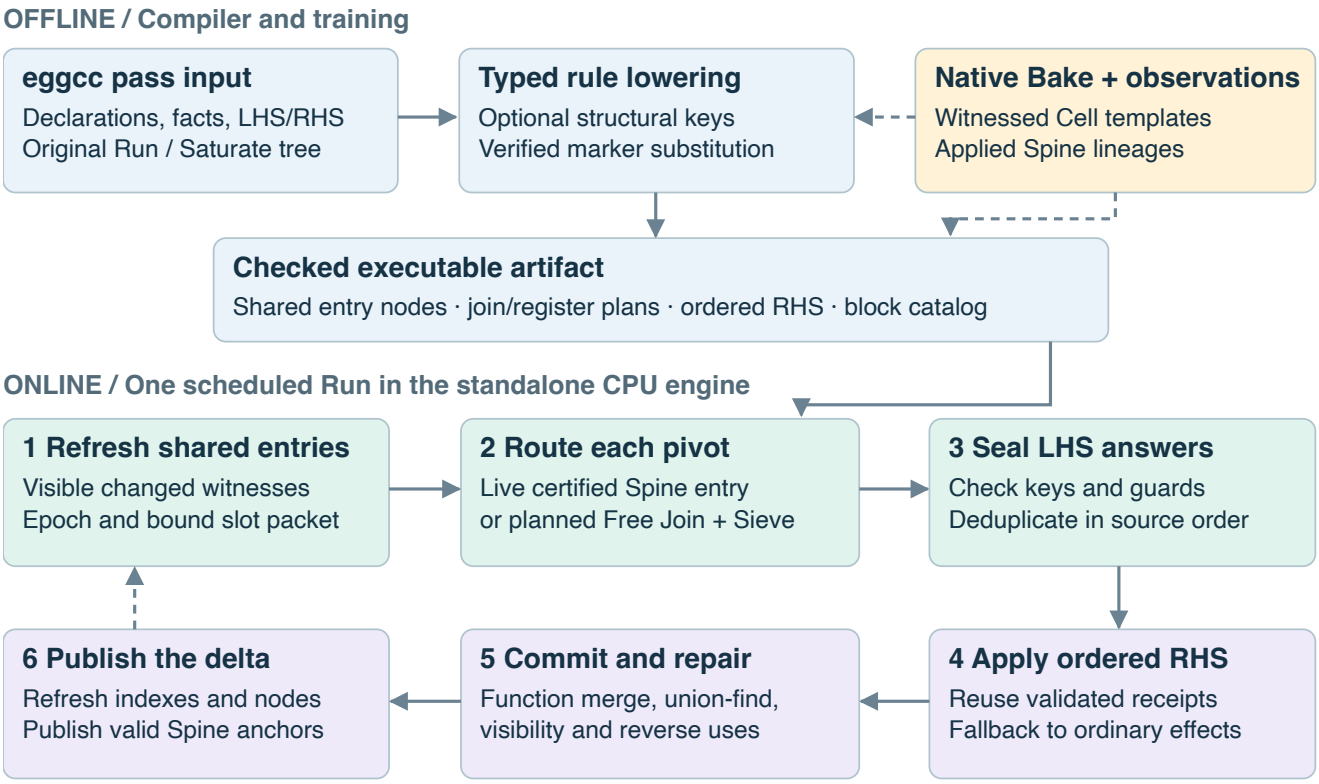


Fig. 1 The compiler produces a checked artifact. The colored runtime path repeats at the caller's next legal Run. Dashed inputs carry training evidence or refreshed state across a boundary. Native extraction sits outside the replay timer.

Two kinds of prior knowledge. The compiler proves facts such as "this key is bound" or "this RHS requests the same constructor as that producer." Training observes how often a path actually wins. The first supplies the legality conditions. The second decides whether spending memory and compilation effort on the path is worthwhile. Compiler entry · Artifact loader

What sharing can save. Suppose d changed rows satisfy one atom shape watched by m consumers. If all m separately decode and test that atom, its entry work is dmc, where c is the cost per decode and test. With a shared entry it is approximately $dc + d\sum_i p_i$, where p_i is the cost of projecting and delivering the packet to consumer i. Each consumer still pays for its residual join, sorting and effects. This local model predicts a saving only when dispatch and maintenance cost less than the redundant entry work. Egglog can already share physical indexes, so that common cost is outside this comparison.

2. From a rule to a reusable plan

SOURCE TRANSFORMATION

The Rust front end reads the flattened pass, installs explicitly requested structural-key selection, runs Bake on a compatible training input, selects markers, and lowers the transformed program to Recipe IR. A marker represents a positive conjunction with explicit ports. The compiler emits maintenance rules to populate it before dependent consumers run. Its selection problem trades saved consumer atoms against maintenance cost, with structural estimates or measured inputs. Observing a Cell during Bake proposes reuse. The source transformation still needs a rule-derived positive view and a legal place in the schedule. Marker selection · Cell witnesses

PHYSICAL LOWERING

A rule becomes typed query atoms, input slots, effect destinations and a source-owned control tree. The compiler enumerates legal pivot-specific orders. A register kernel can admit a connected query with one indexed expansion, subsequent full-key reads and primitives whose operands are bound. Fresh columns write registers. Repeated variables, literals and previously assigned columns generate equality checks. The loader reconstructs physical plans from the typed recipe and checks serialized plans. Kernel derivation · Kernel interpreter

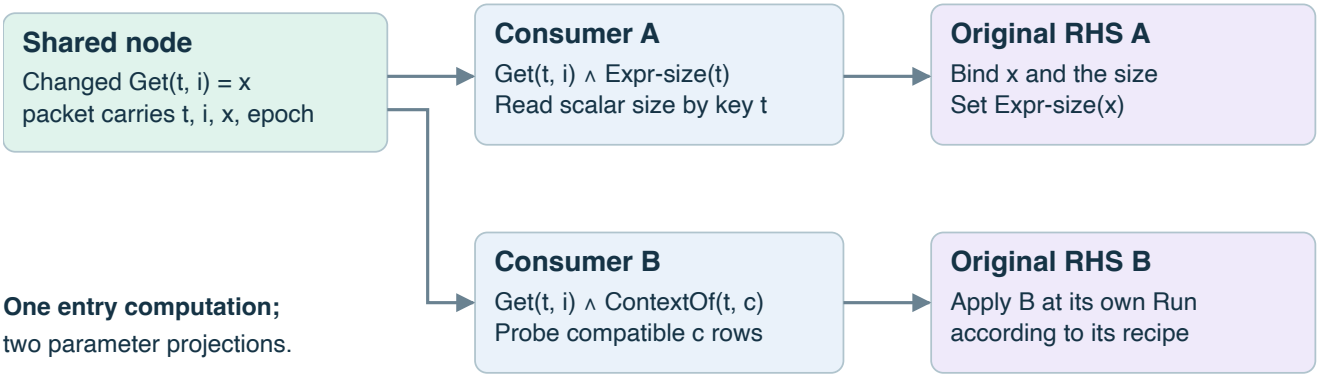


Fig. 2 Two consumers can subscribe to the same normalized atom shape. An arrival supplies a correlated row, its witness and epoch. Consumer projections map packet positions to local slots. An update to the other atom can also be the rule's pivot.

For a changed row r and consumer watermark w, entry selection restricts the delta to visible rows with $r.\text{epoch} \geq w$. The Sieve can prove that necessary domains do not intersect. Planned Free Join can probe compatible keys before materializing seeds and can start at a selective indexed domain. A fingerprint narrows the candidates. The matcher checks full values before emitting a binding. Ownership by epoch, witness and pivot plus source-slot deduplication eliminates duplicate derivations at the Run boundary. Shared nodes · Join routing · Sieve

3. Cells, repair and the Run boundary

A **Cell template** contains an ordered effect recipe. An **instance** records its template ID, bound slots, optional parent and live fact positions. A constructor fact can be named by a `FactRef {cell, op}`. The recipe tells the engine which relation and slot operands that position denotes. Decoding those operands produces the current logical row. A mutable function has a current key, value, owner and epoch. Query indexes and table widths still matter: storing facts through shared recipes trades duplicate representation for decoding, liveness tests and reference maintenance. Graph and repair · Frame storage

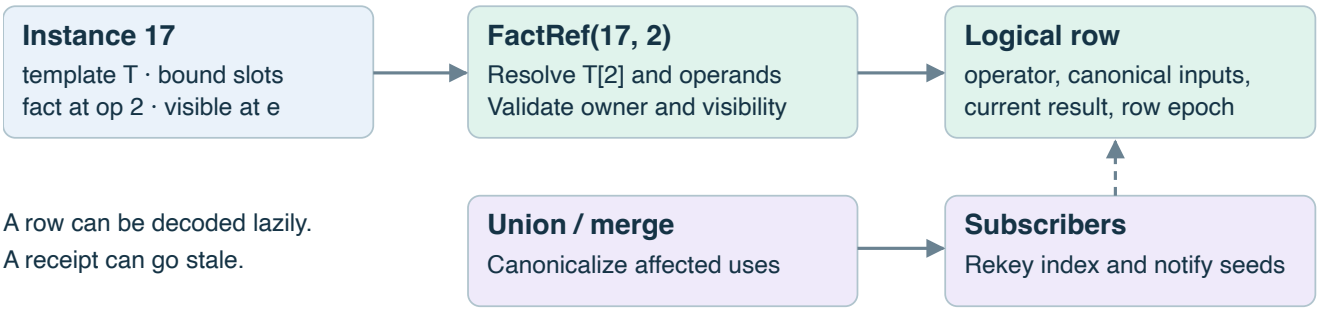


Fig. 3 The address of a constructor row combines a concrete Cell instance and an instruction position. A mutable function row has its own current owner. Purple shows the repair path after a key changes.

Physical object	What consumers may reuse	What can invalidate it
Cell fact position	A committed constructor witness	Deletion, subsumption, movement or a changed canonical key
Function receipt	The current committed function row	An ordered merge or replacement of its current value
Index bucket / packet	Candidate witnesses for a key and epoch	Membership, visibility or representative change
Spine cursor	Bounded producer lineage at a live anchor	A stale receipt or failed consumer precondition

EXECUTION CONTRACT

```
for each Run selected by the source schedule:
    read all rule LHSs against the current snapshot
    seal complete bindings in the rule's original input order
    apply their RHS instructions in order
    repair congruence and rekey affected rows
    publish visible changes, update subscribers and watermarks
    evaluate the next repeat/saturation condition
```

On union, reverse dependencies identify constructor and function keys involving an affected representative. The engine invalidates decoded payloads, handles rekeying or coalescence and delivers changes to the relevant indexes. Consumers must be able to observe old-key removal, new-key arrival, visibility and forwarding. A compiled block does not remove this repair obligation. A cheap RHS path can avoid registration of a temporary frame when a typed single function write can be completed directly. A failure of its preconditions resumes ordinary effects. Run and RHS execution · Storage and kernel ablation

4. How a block crosses Runs

A **Spine** follows committed facts that serve as anchors for subsequent rules. The compiler records which consumer pivot can read a producer's output and whether its remaining ports can be obtained exhaustively. The producer publishes a weak reference when its fact becomes visible. A later consumer can use the reference only at its own scheduled occurrence. The next matched rule can have other LHS entries, other matches and ordinary effects in the same Run.

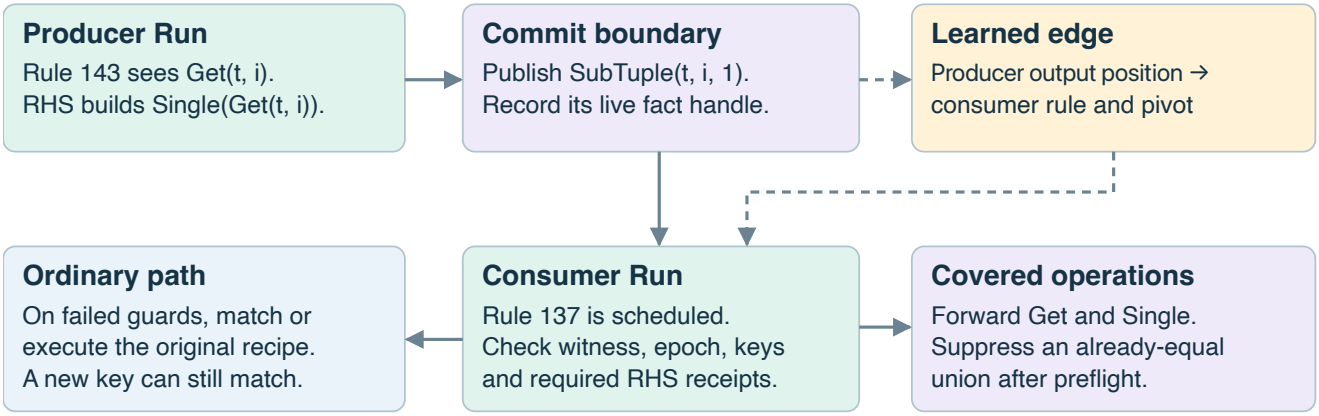


Fig. 4 An observed producer/consumer pair from Raytrace. The published SubTuple fact enables a later pivot. Producer equalities and constructor receipts can also satisfy part of the consumer RHS. The dashed edge requests a switch. The current runtime state decides whether it succeeds.

Concrete saving. Rule 143 witnesses `Get(t,i)`, constructs its `Single` and equates that result with `SubTuple(t,i,1)`. A later rule 137 requires the same constructor key. The compiler records the donor instruction, distance, and consumer destination. The runtime validates the current row and canonical keys before reusing it. A whole-RHS preflight checks every required receipt before any effect is committed. An unexpected replacement chooses the ordinary arm. This avoids failed speculation partially applying a write. Switch construction · Measured pair

Learning larger blocks. Start with 32 applied consumer/pivot leaves. A training round counts witnessed adjacent units on a winning lineage. A proposed binary unit names a left unit, the publishing instruction and a right unit. Freeze the catalog during each run, bound its size, and let an actually applied macro become a later operand. The donor used here has a 64-unit ceiling, at most four primitive steps, eight nodes, up to eight additions per round and a 512-entry candidate budget. When transferring, the harness asks the compiler to resolve each leaf's typed rule signature on the target. It remaps only reachable joins and rebuilds every RHS handoff against that target. Typed leaf transfer · Bounded composition

5. What explains the gain

The strongest historical improvement over the earlier Cell implementation came from compiling fast per-pivot register joins, specializing single writes, and reducing frame and index overhead. The comparison below uses frozen 64-block artifacts and single-thread release builds. The numerator is a previous **Cell** engine. §6 gives the independent egglog comparison. Paired historical measurements

Workload	Earlier Cell	Optimized Cell	Ratio	Pairs above 2×
sum-bits	79.34 ms	37.07 ms	2.14×	8 / 9
Raytrace	33.81 s	15.50 s	2.18×	5 / 5

On Raytrace the register plans produced 2,650,667 bindings and the single-write plans avoided 1,017,798 transient frame registrations. Peak process memory fell from 4,122 to 3,265 MiB in that paired study. The reported candidate-work counter actually rose. A guess about fewer row visits would therefore give the wrong explanation for this result.

What RHS specialization buys separately. With the fixed Raytrace catalog, the later guarded switch avoided 30,896 constructor requests and 17,152 complete RHS frames. Its comparison against dispatch alone was faster in one of four matched rounds. Fewer operations and a demonstrated throughput gain require separate measurements. RHS ablation

Why context retention stays optional. In a five-atom loop-invariant rule, a `DoWhile` root keyed reads of input context, output context and type. Retaining up to 84 correlated prefixes let the final `Const` expansion join against the changed side. Rule search dropped from 149.31 to 50.45 ms and emitted the same 143,282 bindings. Whole-pass Raytrace inference was 13.837 versus 13.844 s. Generalizing the compiler found cuts in 481 of 582 rules. Enabling all cuts increased full-pass visits from 8.42 to 10.33 million relative to narrow retention. The portable path used in this report leaves the context experiments off. Loop-invariant trace · Admission study

Break-even condition. Retaining a prefix pays when avoided probes, decodes and sorting exceed work to observe changes, validate roots, refresh prefixes and route answers. It matters how often old prefixes meet new terminal rows. Counting only distinct roots hides this join multiplicity. Query hints also need a lifetime budget and a fallback for changing workloads. The present single-thread measurements supply no scaling curve for a parallel implementation.

6. Three measurements of eggcc workloads

Production. The capture ran eggcc at `ae80f82` with its pinned `egglog-experimental 08771f9` and actual Tiger extraction feeding the next pass. The pass driver was instrumented. Appendix B sums time inside ordinary Run and outer Saturate packets once, then separately reports extraction and execution including setup. These are archived measurements on original emitted programs. They answer where eggcc spent time. Pinned dependency

Matched replay. For every captured pass the experimental frontend resolved the original program, printed the resulting flat commands, and normalized generated names. Both timed lanes start from that unmarked source. Native Free Join uses the project's immutable `egglog-baseline 62ae6baa`, with an external timing adapter and compiled-out work counters. The native clock starts at the first Run schedule after bootstrap and includes subsequent commands. Eggfuse's Rust compiler transforms the same flat input and emits a marked typed recipe. Its clock covers schedule execution after graph setup. The timings exclude training, parsing, final export and Tiger. Raw experimental syntax needs the experimental frontend; passing it directly to the tag runner fails the prelude/type environment on the first corpus input. Native adapter and counter boundaries

Fixed treatment. One thread, ordinary storage, shared nodes, planned Free Join, Sieve, intersection, range guards, static RHS aliases, Stage Spine and receipts. All targets use one frozen, small marker-training program exercising terms and helpers, and one iterative block donor learned from real sum-bits Pass 3 executions. These supply different compiler inputs. The target receives 59 deployable units in each Pass 1 and Pass 2 program and 64 in each Pass 3 program. Cell setup derives and checks each target's plans. This is within-workload transfer for the donor benchmark and cross-workload deployment elsewhere. The suite uses this policy throughout. Runtime settings, input hashes, repetitions and compiler costs are recorded with the data.

Repeated Pass 3	Pairs	Native ms	Cell ms	Ratio	Cell wins
sum-bits	4	43.7	35.3	1.238×	4
fenwick_tree	4	107.8	84.2	1.280×	4
raytrace	4	23,514.4	13,469.6	1.746×	4

One excluded warm-up per arm, then four alternated fresh-process pairs. Ratios divide the two medians. The corpus survey executes one sample per arm and alternates order by workload index. Close single-sample outcomes should be interpreted at their measured precision.

At the process boundary, native / Cell median wall time was 104.6 / 145.5 ms for sum-bits, 170.3 / 208.7 ms for fenwick_tree, and 23.673 / 16.196 s for Raytrace. The Cell CLI also exports the final graph and detailed report. These observations expose startup and output costs; the two CLIs have different output duties.

Earlier case-specific catalogs produced native-original / Cell inference medians of 46.43 / 36.68 ms on sum-bits and 24.192 / 14.015 s on Raytrace in a separate matched experiment. Native-on-marked medians in that study were 54.85 ms and 21.697 s. The original-program column is the baseline for an architectural claim. The marked column measures how the transformation changes the work egglog sees. The companion archive includes the historical samples.

The upstream baseline on its own terms

The first numeric column below is the time eggcc's pinned engine spent in scheduled packets on the original inputs. Its driver ran each pass, extracted with Tiger, and supplied that result to the next pass. The capture also separates parsing and non-schedule setup. Columns N and C are fresh replays of those captured boundaries. They answer the narrower question of what an alternative execution layer costs on a common input representation. Historical production runs and fresh replays have different harnesses and sampling dates.

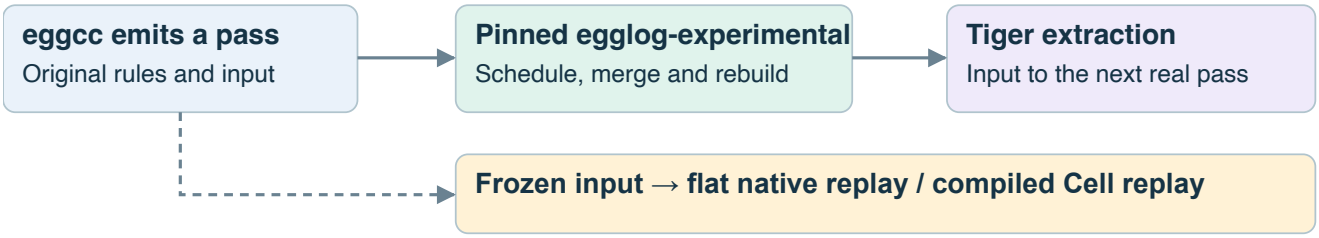


Fig. 5 The solid path is the measured production pipeline. The dashed branch reruns that specific pass input under two execution engines. Replaying the three captured inputs isolates inference from drift in preceding extracted programs.

All workloads	Production sched, s	Tiger wall, s	N replay, s	C replay, s
Pass 1	30.75	7.89	31.15	26.49
Pass 2	64.70	19.01	63.43	61.53
Pass 3	48.97	10.31	48.86	38.79

Production figures cover all archived cases per pass. Replay totals cover completed pairs shown on the first page. The original driver profile does not record an explicit thread setting. Only the fresh N/C comparisons here enforce one thread.

WHAT EGGLOG ALREADY DOES

The tagged engine decomposes a query into join stages. During Free Join execution, a chosen cover enumerates keys or projected rows, other atoms probe indexes, and intersected domains constrain later stages. Completed substitutions reach an action buffer. The rule bridge invokes native merge and rebuild machinery, including incremental versus nonincremental rebuild selection. Shared indexes, an incremental frontier, compiled join plans and vectorized actions are features of the baseline. The proposed extension reuses correlated cross-rule entry packets and physical producer receipts across Run boundaries. Native Free Join · Native rule bridge and rebuild

A COUNTER IS A UNIT OF WORK

On historical frozen sum-bits, instrumented native Free Join on the original source recorded 40,425 enumerated index keys, 5,918 projected scan rows, 871 materializations and 34,698 probes. The 8,763 bindings delivered to the action buffer were counted separately. Eggfuse's visits sum direct candidates, residual witnesses or key attempts, merge and Spine candidates and shared predicate checks. Comparing the two integers as numbers of rows eliminated would mix different operations. Timings here use the native runner with counters compiled out. Definitions and native ablation

The storage proposal can be evaluated incrementally. First keep native Free Join and table storage as the oracle and share one entry computation for consumers with identical normalized atom shapes. Then compare full bindings from register kernels at the action-buffer interface. Introduce a committed fact handle only for operations that can prove its identity, visibility and canonical key. This sequence isolates which interface earns its implementation cost.

7. Semantic boundary and upstream decision

The current within-engine audits compare ordered RHS inputs, complete schedule boundaries, committed operation counts, final logical graphs and winning Spine lineages against frozen Cell references. Sum-bits has 1,428 audited Runs, 3,034 resumed steps and maximum depth 12. Raytrace has 16,883 Runs, 663,403 resumed steps and depth 29. These tests catch an illegal early activation or a dropped binding between Cell configurations. Cross-engine replacement also needs comparison against egglog.

> **A reproducible difference.** On sum-bits, both the original native export and current fixed-64 Cell export have 3,206 rows after projection to original table names; all per-table counts agree. Grounded reconstruction nevertheless finds a visible Cell `TermSingle` with no matching native key. All 19 constructor occurrences in the witness resolve to visible Cell rows. Appending the ground check to the original flat source gives `CheckError` in the timed baseline binary. The same check fails on the marked input in native egglog. The archive includes the concrete term, both exports and replay commands.

The diverging term can be written compactly with `g(i) = TermGet(TermArg(), i)` and `b(op,x,y) = TermBop(op(),x,y)`:

```
q = b(Div, g(1), g(2))
p = b(Mul, q, TermConst(Int(2)))
witness = TermSingle(b(Add, g(0), b(Sub, g(1), p)))
```

This locates an observable membership difference. It leaves open whether the changed graph produces a different Tiger-extracted program and which implementation decision first caused the divergence. In particular, an order-sensitive merge can depend on an earlier schedule or binding-order difference. The table below identifies a smaller upstream experiment that would distinguish these causes.

Proposed seam	Retain in egglog	Acceptance evidence
Compiled LHS entry	Native Run snapshot, watermark and action-buffer order	Shadow each pivot and compare complete substitutions, including misses and ties
Register / writer plans	Native value interning, conflict resolution and application order	Compare key reads and ordered effects for insert, replace and no-op
Receipt publication	Native rebuild, subsumption and canonicalization	Invalidate on old/new keys, visibility and forwarding; test deleted and merged witnesses
Cross-Run block	Native scheduling and extraction	Check guards against current rows. Compare grounded queries and extracted IR through all passes

The first implementation milestone is an opt-in alternative matcher feeding egglog's ordinary action buffer. Run the exact counterexample as a regression and capture the first divergent source Run. Only after equality partitions, function outputs, visibility and selected terms agree should a production trial replace native effect or storage handling. A final trial should include offline cost amortization, total compiler wall time, peak memory and execution of the resulting binaries.

8. Distribution and cost of portability

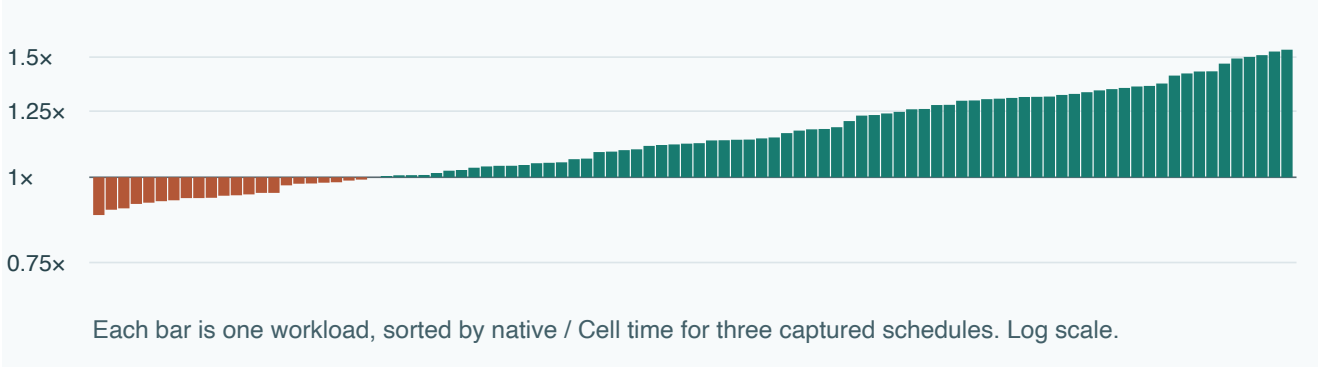


Fig. 6 Equal-height bars correspond to equal multiplicative changes. The horizontal reference is parity. All cases contribute one observation irrespective of program size.

Three-pass tails	Native / Cell	Native sum, ms	Cell sum, ms
norm.bril	0.880×	389.3	442.3
3mm.bril	0.896×	5,399.5	6,024.4
conjugate-gradient.bril	0.900×	1,259.6	1,399.0
fitsinside.bril	1.538×	30.7	20.0
fact.bril	1.529×	53.6	35.1
raytrace.rs	1.510×	40,636.1	26,910.3

The first three rows are the largest regressions among the covered tasks; the last three are the largest improvements. Reporting them alongside the aggregate makes the deployment trade-off visible. The geomean weights one workload once. The time-weighted ratio answers how long the entire sequential sweep spends in inference. Both use sums of three frozen inference intervals per case.

Interpreting preparation. The general compiler re-runs the fixed marker Bake, selects markers, lowers the target, then validates and links the donor blocks for each independent artifact. This deliberately straightforward driver repeats preparation that a service might cache. It does not measure such a cache. If all measured preparation is additional and startup/export differences are set to zero, the illustrative break-even reuse count is preparation time divided by positive per-execution inference savings. Over this suite the implied break-even is about 48 replays of the same prepared artifacts. Compiling each program once for a single inference run would lose the observed 16.62 s saving many times over. Sharing plans across input programs is therefore an integration requirement worth measuring. When inference is slower, repeated runs do not repay preparation.

Reproducibility. The data package contains the 288 flat programs, raw eggcc inputs, compact production summaries, the frozen donor, full per-run JSONL and CSV, and Cargo-backed replay code. Keys are workload path plus pass number. Duplicate basenames remain distinct. Source links point to the tested commit. The unpinned workstation had concurrent system activity. Alternating order and repeating three anchors reduces some timing noise; the full-corpus single samples have no per-task confidence interval. Automatic context cuts, reference views, local saturation, union grafts and GPU execution were outside this fixed policy. The suite provides evidence about this pinned revision and machine. Other egglog releases, architectures and thread counts require their own measurements.

9. Memory and cross-Run lifetime

6 NOVEMBER 2026 / TWO-WORKLOAD UPDATE

This follow-up measures the standalone Cell CPU engine at `27f098a2` and the lifetime changes implemented in `4d9ea942`. It uses the frozen sum-bits and Raytrace Pass 3 artifacts, their original 64-unit Bake catalog, one CPU thread and ordinary release builds. These runs add evidence about memory to the earlier suite. Their timing matrix has its own control; no PGO is used here.

Pass 3	Control ms	Reclaimed ms	Control MiB	Reclaimed MiB
sum-bits	19.158	18.740	152.18	146.81
raytrace	6,746.279	6,716.853	1,856.79	1,432.68

Raytrace's peak RSS fell by **22.8%**. In the same matrix, complete CLI medians were 10.373 and 10.374 s. The median of paired inference differences was +16.663 ms, with only one of four reclaimed runs faster. A lower ratio of medians therefore supplies little evidence of a latency improvement.

RECHECKING THE NATIVE MEMORY GAP

Two new runs of eggcc's pinned native egglog `ebba7bb` executed the original unmarked Raytrace source with a verified one-worker pool. macOS `time -l` recorded 462.375 and 450.047 MiB peak RSS; the median was 456.21 MiB. Native timing included parsing, initialization and inference, without a graph export. Cell RSS was sampled after inference and before its export, including its artifact loader and generated tables. The control/native memory ratio was approximately **4.07×**, falling to **3.14×** with reclamation. Source and binary hashes accompany the measurements.

COUNT LIVE ALLOCATIONS ONCE

A diagnostic allocator attributes requested bytes to the allocating operation and subtracts them on deallocation. A shared Rc allocation is charged once. Reallocation retains its original category. At Raytrace inference completion it counted **1,762.98 MiB** still live.

Allocation origin	Live requested MiB
graph	494.6
artifact	378.5
search	215.4
columns	163.4
entries	149.0
spine	121.3
other	77.0
row payload	76.9

The largest remaining categories include RHS (62.6 MiB), Sieve (22.1 MiB) and captured history (2.3 MiB). Allocation origin can differ from the eventual owner: matched parameters can become persistent Cell slots. Category peaks occur at different times. The ledger excludes allocator fragmentation and C allocations; its instrumented timing and RSS are excluded from the speed comparison.

Evidence: `evidence/memory-lifetimes.json`, rendered by `reproduce/render-memory-data.py`. The original 96-workload tables retain their earlier source revision and measurement dates.

Consumers determine retention

A shared entry node owns a set of declared rule consumers. After a source ruleset occurrence, each rule's logical watermark advances, including rules omitted by the ready set. The node can release a materialized packet once its epoch is **strictly below every consumer's watermark**. An equal epoch can still seed a match. A rule appearing in several sets advances with its latest actual occurrence.

The implementation keeps a compact membership header so join planning sees the same cardinality. A changed fact can rebuild its packet through the existing publication stream. An explicitly older watermark request marks the node cold and reconstructs its visible rows. Reclamation can run immediately or conservatively every 32 graph epochs. The latter delays release without advancing the readers.

On Raytrace, immediate reclamation reduced retained entry packets from **665,584** to **82,355**. Entry-origin requested storage fell from 149.0 to 72.4 MiB. There were 882,328 retirements over the schedule, including repeated incarnations of some witnesses.

COMPLETION RECEIPTS NEED FEWER PRODUCER OWNERS

The final Raytrace Cell graph has 1,034,514 logical rows. Generated `CellSpineView*` tables account for 213,334; `CellSpineApplied*` tables account for 121,502. Together they contribute **32.4**% of the rows. This table-name grouping describes the measurement. The optimization uses executable read contracts.

The three populated Applied tables have no declared query or RHS readers. Their Unit values use an old-value merge. The compiler derives origin demand from all rule queries, executable table reads and calls in merge expressions. It checks an emitted certificate against the target program. An unread receipt can keep its key, value and rebuild identity while releasing its optional creating-frame reference. An assertion or later declared rule that reads the table retains that producer.

Pass 3	Full live Cells	Embedded snapshots	Both forms
sum-bits	2,385 → 2,177	490 → 639	2,875 → 2,816
raytrace	601,924 → 485,196	85,989 → 163,541	687,913 → 648,737

Some remaining producers change from full Cells into embedded snapshots after an unnecessary second owner is removed. The combined count measures that distinction. Every completion-table row stays in the logical export. A caller choosing the new mode can receive `None` from the optional origin accessor for an unread receipt.

RELEASE VALIDATION IR AFTER INITIALIZATION

A teardown probe freed about 323.7 MiB by dropping the original Raytrace validation JSON, plus 17.1 MiB for its input buffer. The explicit `--release-source-artifact` option releases these after runtime plans are initialized. Typed plans, structural-key nodes and the original source SHA remain available. The residual object has a provenance-only schema, which the source loader rejects. Offline Bake or revalidation requires reloading the original artifact.

For sum-bits, the combined live-requested footprint fell from 131.99 to 22.08 MiB, while peak RSS changed by much less. Its parser/setup high-water had already occurred. This separates the benefit of reclaiming completed input state from the cost of initially loading it.

Cost, correctness and remaining scope

The primary Raytrace matrix isolates entry retirement from validation-IR release.

Arm	Inference s	Peak RSS MiB
Control	6.822	1,864.23
Consumed entries	6.813	1,788.11
Validation IR	6.816	1,549.02
Both	6.719	1,475.88

The final matrix combines all three changes, with 32-epoch retirement. It rotates eight processes per sum-bits arm and four per Raytrace arm. Compilation and diagnostic runs are outside the measurements.

Pass 3	Paired delta	Faster pairs	Control CLI ms	Reclaimed CLI ms
sum-bits	-0.362 ms	6/8	116.665	116.369
raytrace	+16.663 ms	1/4	10,372.988	10,374.331

The primary sum-bits retirement-only arm was slower in all eight pairs. The evidence supports a memory reduction with a small, workload-dependent maintenance cost. Both lifetime features remain opt-in.

AUDIT THE SAME SCHEDULED OBSERVATIONS

Full audits matched the frozen graph, schedule, ordered RHS inputs, effective writes and Spine trajectory. They include the combined mode and forced fingerprint/Sieve collisions. Twenty-eight runtime tests and fifteen compiler tests passed, covering reader lifetimes, backward watermark requests and rejected certificates.

These are within-Cell checks. The grounded cross-engine witness in §7 requires its own resolution. The memory comparison alone establishes neither native semantic equivalence nor a completed twofold speed target.

WHAT CROSS-RUN COMPLETION CAN STILL ABSORB

History-reference trimming can recover only a few MiB in this measurement. Larger structures hold graph records, parameters, indexes and shared entry state. Future blocks should expose reader demand and last-use ports, then measure reconstruction costs against another retained row copy.

Generated view and completion rows are the next candidate. A compressed form must still support required canonical-key lookups, union forwarding, last-writer publication and exports. Their generated origin alone cannot justify removal.

> **Reproduce the update.** `reproduce/memory-replay.sh` accepts checkout `4d9ea942` and two frozen program files. Features `entry-retirement` and `origin-demand` select lifetime optimizations; `--release-source-artifact` releases validation IR. Bundled data contains all three matrices and their hashes.

APPENDIX A / FROZEN REPLAY

Per-workload inference

Milliseconds. Within each pass, N denotes native egglog on unmarked flat input and C denotes the standalone Cell executor. The final column divides the three-pass sums. LA abbreviates *linear-algebra* in the paths.

ID	eggcc passing workload	P1 N	P1 C	P2 N	P2 C	P3 N	P3 C	N/C
0	bril/core/ackermann.bril	17.5	13.8	11.1	9.5	27.3	18.7	1.33×
1	bril/core/armstrong.bril	130.3	84.2	84.2	65.8	69.8	66.5	1.31×
2	bril/core/binary-fmt.bril	40.0	26.2	10.1	7.8	16.2	14.4	1.37×
3	bril/core/birthday.bril	42.4	30.1	47.1	32.2	13.6	9.9	1.43×
4	bril/core/bitshift.bril	28.8	20.4	11.8	10.8	25.7	27.2	1.14×
5	bril/core/bitwise-ops.bril	120.1	108.7	187.0	160.6	273.1	348.1	0.94×
6	bril/core/catalan.bril	36.7	28.4	64.1	45.3	39.6	33.3	1.31×
7	bril/core/check-primes.bril	84.9	76.0	62.6	48.9	45.2	46.9	1.12×
8	bril/core/collatz.bril	67.9	50.0	19.6	15.9	29.7	29.3	1.23×
9	bril/core/digital-root.bril	71.1	59.0	17.5	15.0	59.6	58.6	1.12×
10	bril/core/dot-product.bril	21.4	16.2	20.0	15.4	40.2	28.3	1.36×
11	bril/core/euclid.bril	30.6	19.3	53.6	40.9	25.2	17.5	1.41×
12	bril/core/fact.bril	35.0	21.8	7.6	5.3	10.9	8.0	1.53×
13	bril/core/factors.bril	64.2	44.2	72.2	53.8	32.6	28.0	1.34×
14	bril/core/fitsinside.bril	9.7	6.4	9.2	6.0	11.8	7.5	1.54×
15	bril/core/fizz-buzz.bril	84.7	60.4	62.8	45.5	39.9	35.5	1.32×
16	bril/core/gcd.bril	60.5	47.4	17.8	16.0	36.0	33.7	1.18×
17	bril/core/hanoi.bril	60.8	40.5	13.2	10.2	21.1	21.3	1.32×
18	bril/core/is-decreasing.bril	50.7	36.6	17.1	14.1	58.8	47.1	1.30×
19	bril/core/lcm.bril	49.4	34.7	14.5	10.8	43.9	36.7	1.31×
20	bril/core/loopfact.bril	38.8	26.2	37.9	25.2	11.3	7.6	1.49×
21	bril/core/mod_inv.bril	40.8	27.6	82.4	64.9	74.9	71.5	1.21×
22	bril/core/orders.bril	88.6	75.7	26.1	32.3	95.0	120.5	0.92×
23	bril/core/palindrome.bril	127.5	113.0	128.5	133.9	128.4	134.9	1.01×

APPENDIX A / FROZEN REPLAY

Per-workload inference

Milliseconds. Within each pass, N denotes native egglog on unmarked flat input and C denotes the standalone Cell executor. The final column divides the three-pass sums. LA abbreviates *linear-algebra* in the paths.

ID	eggcc passing workload	P1 N	P1 C	P2 N	P2 C	P3 N	P3 C	N/C
24	bril/core/pascals-row.bril	52.7	35.3	59.4	43.3	23.7	16.3	1.43×
25	bril/core/perfect.bril	93.3	69.3	20.4	18.0	68.1	55.1	1.28×
26	bril/core/primes-between.bril	133.0	117.7	119.0	97.1	124.9	118.0	1.13×
27	bril/core/pythagorean_triple.bril	84.4	74.9	24.1	21.3	34.7	30.3	1.13×
28	bril/core/quadratic.bril	65.0	48.9	18.4	16.0	49.9	42.0	1.25×
29	bril/core/recfact.bril	36.3	22.6	7.4	5.2	10.3	8.2	1.50×
30	bril/core/relative-primes.bril	54.1	45.8	57.8	42.2	38.5	40.4	1.17×
31	bril/core/reverse.bril	65.8	51.6	73.8	60.4	47.4	38.8	1.24×
32	bril/core/sum-bits.bril	28.0	19.1	14.0	10.2	57.2	38.3	1.47×
33	bril/core/sum-check.bril	50.0	38.7	60.4	43.3	25.6	19.0	1.35×
34	bril/core/sum-divisors.bril	100.7	78.9	26.5	28.8	72.4	82.5	1.05×
35	bril/core/sum-sq-diff.bril	50.4	42.1	61.3	57.5	26.7	26.3	1.10×
36	bril/core/totient.bril	110.4	79.2	96.4	84.3	52.8	42.8	1.26×
37	bril/core/up-arrow.bril	43.3	29.8	85.7	69.1	109.5	115.6	1.11×
38	bril/float/conjugate-gradient.bril	250.3	315.9	524.7	592.3	484.6	490.7	0.90×
39	bril/float/cordic.bril	458.0	339.8	383.2	403.5	885.0	831.8	1.10×
40	bril/float/euler.bril	29.7	19.9	46.3	33.5	17.3	15.3	1.36×
41	bril/float/leibniz.bril	76.2	53.2	76.6	57.1	31.7	31.0	1.31×
42	bril/float/mandelbrot.bril	137.4	109.9	232.4	223.5	111.1	119.0	1.06×
43	bril/float/n_root.bril	68.0	55.7	97.3	91.7	74.5	80.6	1.05×
44	bril/float/newton.bril	34.8	24.0	53.3	39.9	43.4	37.2	1.30×
45	bril/float/norm.bril	99.2	91.1	152.3	162.9	137.8	188.2	0.88×
46	bril/float/ray-sphere-intersection.bril	16.6	12.5	15.7	12.1	24.3	29.8	1.04×
47	bril/float/riemann.bril	48.7	44.7	22.8	23.3	85.9	98.8	0.94×

APPENDIX A / FROZEN REPLAY

Per-workload inference

Milliseconds. Within each pass, N denotes native egglog on unmarked flat input and C denotes the standalone Cell executor. The final column divides the three-pass sums. LA abbreviates *linear-algebra* in the paths.

ID	eggcc passing workload	P1 N	P1 C	P2 N	P2 C	P3 N	P3 C	N/C
48	bril/float/sqrt.bril	67.2	56.6	68.5	55.7	40.7	37.8	1.18×
49	bril/long/dead-branch.bril	52.8	37.7	52.2	38.0	29.3	18.9	1.42×
50	bril/mem/adj2csr.bril	161.7	150.0	278.5	304.6	159.6	193.7	0.93×
51	bril/mem/adler32.bril	88.8	76.7	33.3	32.6	118.0	105.1	1.12×
52	bril/mem/binary-search.bril	51.3	43.9	80.3	64.0	66.4	65.7	1.14×
53	bril/mem/bubblesort.bril	98.1	79.1	163.7	163.6	108.3	97.3	1.09×
54	bril/mem/csrmv.bril	314.0	292.0	610.8	687.4	610.2	543.8	1.01×
55	bril/mem/fib.bril	43.3	27.7	90.5	72.0	37.5	27.1	1.35×
56	bril/mem/major-elm.bril	60.4	44.0	34.2	27.6	113.7	91.6	1.28×
57	bril/mem/mat-mul.bril	146.2	126.4	256.5	267.1	145.1	146.4	1.01×
58	bril/mem/max-subarray.bril	68.0	46.5	120.4	89.7	71.0	64.1	1.29×
59	bril/mem/quickselect.bril	99.0	84.2	168.8	157.9	164.5	193.6	0.99×
60	bril/mem/quicksort.bril	138.2	95.4	167.6	165.0	367.7	385.7	1.04×
61	bril/mem/vsmul.bril	45.5	33.3	88.4	68.7	53.0	46.4	1.26×
62	bril/mixed/cholesky_decomp.bril	361.0	315.7	519.7	588.4	438.2	525.7	0.92×
63	bril/mixed/mat-inv.bril	121.5	94.8	126.3	86.9	187.7	171.3	1.23×
64	fenwick/fenwick_tree.rs	133.2	115.3	194.3	207.6	111.7	100.6	1.04×
65	polybench/datamining/correlation.rs	1,149.2	992.9	2,557.0	2,738.5	1,063.7	1,139.5	0.98×
66	polybench/datamining/covariance.rs	727.9	765.5	1,865.1	1,936.2	655.7	722.6	0.95×
67	polybench/LA/blas/gemm.bril	695.9	596.2	1,266.8	1,450.4	546.0	644.8	0.93×
68	polybench/LA/blas/gemver.bril	884.5	694.4	1,910.8	2,201.9	720.2	619.8	1.00×
69	polybench/LA/blas/gesummv.bril	322.8	262.1	615.3	595.2	283.1	262.6	1.09×
70	polybench/LA/blas/symm.bril	574.9	465.6	1,229.5	1,048.6	554.4	477.9	1.18×
71	polybench/LA/blas/syr2k.bril	519.8	446.3	1,029.1	1,033.8	445.3	421.9	1.05×

APPENDIX A / FROZEN REPLAY

Per-workload inference

Milliseconds. Within each pass, N denotes native egglog on unmarked flat input and C denotes the standalone Cell executor. The final column divides the three-pass sums. LA abbreviates *linear-algebra* in the paths.

ID	eggcc passing workload	P1 N	P1 C	P2 N	P2 C	P3 N	P3 C	N/C
72	polybench/LA/blas/syrk.bril	438.1	375.9	767.4	859.9	361.3	357.6	0.98×
73	polybench/LA/blas/trmm.bril	378.1	329.7	651.5	670.5	303.6	290.6	1.03×
74	polybench/LA/kernels/2mm.bril	1,297.2	1,014.9	2,010.8	2,563.5	981.5	980.0	0.94×
75	polybench/LA/kernels/3mm.bril	1,759.4	1,409.3	2,482.2	3,328.4	1,158.0	1,286.7	0.90×
76	polybench/LA/kernels/atax.bril	404.3	341.2	776.6	896.1	301.9	353.5	0.93×
77	polybench/LA/kernels/bicg.bril	378.0	322.2	779.8	815.3	499.3	456.8	1.04×
78	polybench/LA/kernels/doitgen.bril	654.6	597.8	1,123.7	1,126.7	540.3	577.6	1.01×
79	polybench/LA/kernels/mvt.bril	390.4	367.4	811.3	797.8	316.9	320.1	1.02×
80	polybench/LA/solvers/cholesky.bril	594.5	565.5	912.9	970.6	589.1	599.1	0.98×
81	polybench/LA/solvers/durbin.bril	261.4	235.6	532.7	582.2	199.2	197.3	0.98×
82	polybench/LA/solvers/gramschmidt.bril	655.1	574.5	1,448.1	1,299.0	558.4	513.4	1.12×
83	polybench/LA/solvers/lu.bril	510.2	513.7	802.9	932.8	492.1	528.7	0.91×
84	polybench/LA/solvers/ludcmp.bril	915.4	872.1	1,485.3	1,759.2	887.8	835.0	0.95×
85	polybench/LA/solvers/trisolv.bril	186.3	166.8	405.7	423.7	210.0	192.1	1.02×
86	polybench/medley/deriche.rs	2,200.7	1,652.6	4,547.2	5,057.3	2,012.9	2,013.9	1.00×
87	polybench/medley/floyd-warshall.rs	256.4	252.9	659.1	702.0	441.7	417.6	0.99×
88	polybench/medley/nussinov.rs	424.0	368.9	820.2	860.6	691.0	587.6	1.07×
89	polybench/stencils/adi.rs	764.1	690.3	2,624.2	2,169.1	760.6	715.1	1.16×
90	polybench/stencils/fdtd-2d.rs	1,233.5	1,139.7	3,490.9	3,707.6	1,064.5	1,101.6	0.97×
91	polybench/stencils/heat-3d.rs	1,215.6	1,006.6	3,843.5	2,861.6	1,129.9	879.2	1.30×
92	polybench/stencils/jacobi-1d.rs	170.2	161.1	433.7	489.1	147.2	154.8	0.93×
93	polybench/stencils/jacobi-2d.rs	473.2	464.2	1,575.2	1,296.1	416.4	394.6	1.14×
94	polybench/stencils/seidel-2d.rs	336.8	315.0	1,112.0	939.7	324.9	308.1	1.13×
95	raytrace/raytrace.rs	4,733.6	3,919.0	12,394.0	9,214.6	23,508.5	13,776.7	1.51×

APPENDIX B / PRODUCTION EGGCC CAPTURE

Native schedules and Tiger extraction

Milliseconds in eggcc's original captured production run. P1–P3 are scheduled execution only. Extract is the three-pass sum of extraction wall time. Exec is the sum of recorded pass execution time including setup. Keep these columns separate from the fresh replay ratios in Appendix A.

ID	eggcc passing workload	P1	P2	P3	Extract	Exec
0	bril/core/ackermann.bril	15.3	9.8	16.6	40.0	158.7
1	bril/core/armstrong.bril	110.4	69.4	55.6	76.0	350.9
2	bril/core/binary-fmt.bril	33.2	8.2	14.8	32.8	168.2
3	bril/core/birthday.bril	38.6	39.2	14.6	38.2	212.8
4	bril/core/bitshift.bril	25.4	11.0	22.1	42.1	178.0
5	bril/core/bitwise-ops.bril	107.3	172.6	243.3	176.9	657.5
6	bril/core/catalan.bril	30.6	49.7	33.6	51.7	232.9
7	bril/core/check-primes.bril	71.2	50.8	38.7	62.3	274.4
8	bril/core/collatz.bril	61.2	17.0	29.0	45.4	231.4
9	bril/core/digital-root.bril	73.9	15.3	50.3	58.0	262.6
10	bril/core/dot-product.bril	18.0	16.3	34.1	41.7	182.2
11	bril/core/euclid.bril	27.8	49.5	24.7	41.1	236.8
12	bril/core/fact.bril	27.8	6.2	11.6	27.9	155.8
13	bril/core/factors.bril	52.8	55.0	27.7	51.6	245.7
14	bril/core/fitsinside.bril	8.6	7.5	11.1	24.1	137.2
15	bril/core/fizz-buzz.bril	73.9	52.8	31.4	57.4	274.0
16	bril/core/gcd.bril	51.7	13.9	28.2	44.2	211.3
17	bril/core/hanoi.bril	53.9	11.9	22.1	46.0	209.1
18	bril/core/is-decreasing.bril	47.5	16.0	51.7	49.7	249.2
19	bril/core/lcm.bril	40.1	11.5	37.1	39.4	199.8
20	bril/core/loopfact.bril	35.8	32.2	13.5	34.7	204.3
21	bril/core/mod_inv.bril	34.7	65.5	61.8	57.7	281.0
22	bril/core/orders.bril	80.2	22.7	91.9	91.7	320.5
23	bril/core/palindrome.bril	118.3	118.3	121.4	135.7	481.6

APPENDIX B / PRODUCTION EGGCC CAPTURE

Native schedules and Tiger extraction

Milliseconds in eggcc's original captured production run. P1–P3 are scheduled execution only. Extract is the three-pass sum of extraction wall time. Exec is the sum of recorded pass execution time including setup. Keep these columns separate from the fresh replay ratios in Appendix A.

ID	eggcc passing workload	P1	P2	P3	Extract	Exec
24	bril/core/pascals-row.bril	46.4	53.4	22.6	47.0	257.8
25	bril/core/perfect.bril	111.8	17.1	59.8	59.8	312.6
26	bril/core/primes-between.bril	123.2	104.4	104.8	109.7	452.9
27	bril/core/pythagorean_triple.bril	87.8	20.4	32.8	106.7	272.4
28	bril/core/quadratic.bril	60.5	17.5	44.1	51.8	241.7
29	bril/core/recfact.bril	31.3	8.0	12.4	28.9	175.0
30	bril/core/relative-primes.bril	47.6	47.4	36.3	58.5	250.2
31	bril/core/reverse.bril	55.3	61.1	38.9	57.2	270.6
32	bril/core/sum-bits.bril	24.3	12.4	39.1	38.1	192.4
33	bril/core/sum-check.bril	45.6	47.2	26.4	47.8	235.5
34	bril/core/sum-divisors.bril	85.6	21.4	71.5	73.4	295.8
35	bril/core/sum-sq-diff.bril	43.8	52.2	24.2	54.7	238.7
36	bril/core/totient.bril	89.5	85.4	41.6	82.4	336.8
37	bril/core/up-arrow.bril	35.9	76.5	98.1	71.1	326.7
38	bril/float/conjugate-gradient.bril	242.6	524.8	455.5	415.6	1,362.3
39	bril/float/cordic.bril	438.3	361.0	832.2	517.8	1,764.5
40	bril/float/euler.bril	30.1	42.3	20.1	85.8	220.4
41	bril/float/leibniz.bril	68.4	70.1	31.2	58.6	295.2
42	bril/float/mandelbrot.bril	129.5	224.8	104.4	159.8	585.6
43	bril/float/n_root.bril	63.0	89.3	66.3	81.9	342.3
44	bril/float/newton.bril	32.1	50.2	42.1	51.1	250.2
45	bril/float/norm.bril	95.8	135.7	123.3	148.8	483.4
46	bril/float/ray-sphere-intersection.bril	15.3	13.8	23.3	40.6	171.1
47	bril/float/riemann.bril	42.9	19.4	71.8	76.2	253.2

APPENDIX B / PRODUCTION EGGCC CAPTURE

Native schedules and Tiger extraction

Milliseconds in eggcc's original captured production run. P1–P3 are scheduled execution only. Extract is the three-pass sum of extraction wall time. Exec is the sum of recorded pass execution time including setup. Keep these columns separate from the fresh replay ratios in Appendix A.

ID	eggcc passing workload	P1	P2	P3	Extract	Exec
48	bril/float/sqrt.bril	62.9	60.0	36.4	63.9	282.7
49	bril/long/dead-branch.bril	46.6	43.9	28.0	41.0	236.3
50	bril/mem/adj2csr.bril	150.8	259.9	136.8	201.3	678.1
51	bril/mem/adler32.bril	84.3	28.2	104.7	91.9	343.1
52	bril/mem/binary-search.bril	45.7	71.6	56.7	79.5	294.4
53	bril/mem/bubblesort.bril	84.6	145.5	92.8	114.5	444.3
54	bril/mem/csrmv.bril	307.7	582.7	562.4	414.7	1,589.6
55	bril/mem/fib.bril	37.9	76.8	33.7	52.3	266.2
56	bril/mem/major-elm.bril	58.0	30.2	103.5	69.0	319.8
57	bril/mem/mat-mul.bril	142.8	255.1	141.2	199.2	679.0
58	bril/mem/max-subarray.bril	58.9	104.6	66.7	83.1	353.5
59	bril/mem/quickselect.bril	94.9	157.5	147.8	153.8	530.9
60	bril/mem/quicksort.bril	122.4	146.4	317.2	170.3	705.5
61	bril/mem/vsmul.bril	41.9	80.1	46.4	66.4	290.4
62	bril/mixed/cholesky_decomp.bril	351.3	482.9	412.4	394.3	1,378.9
63	bril/mixed/mat-inv.bril	108.0	109.9	154.6	132.5	494.1
64	fenwick/fenwick_tree.rs	118.0	171.0	98.1	129.5	508.7
65	polybench/datamining/correlation.rs	1,077.8	2,494.0	973.4	1,218.4	4,690.4
66	polybench/datamining/covariance.rs	742.7	1,986.4	650.8	914.7	3,532.2
67	polybench/LA/blas/gemm.bril	727.5	1,353.9	547.4	797.5	2,778.8
68	polybench/LA/blas/gemver.bril	904.2	2,039.3	720.2	1,048.1	3,818.3
69	polybench/LA/blas/gesummv.bril	321.4	628.8	275.7	369.0	1,362.8
70	polybench/LA/blas/symm.bril	550.3	1,194.3	554.6	584.5	2,438.4
71	polybench/LA/blas/syr2k.bril	494.9	999.0	444.7	568.1	2,078.2

APPENDIX B / PRODUCTION EGGCC CAPTURE

Native schedules and Tiger extraction

Milliseconds in eggcc's original captured production run. P1–P3 are scheduled execution only. Extract is the three-pass sum of extraction wall time. Exec is the sum of recorded pass execution time including setup. Keep these columns separate from the fresh replay ratios in Appendix A.

ID	eggcc passing workload	P1	P2	P3	Extract	Exec
72	polybench/LA/blas/syrk.bril	434.2	746.1	330.7	461.7	1,643.1
73	polybench/LA/blas/trmm.bril	381.5	655.9	299.3	413.3	1,473.0
74	polybench/LA/kernels/2mm.bril	1,308.1	2,056.7	953.5	1,249.0	4,473.7
75	polybench/LA/kernels/3mm.bril	1,712.2	2,509.9	1,164.1	1,610.7	5,547.5
76	polybench/LA/kernels/atax.bril	380.1	728.0	273.9	444.1	1,512.9
77	polybench/LA/kernels/bicg.bril	366.7	788.5	476.8	472.9	1,770.6
78	polybench/LA/kernels/doitgen.bril	681.8	1,169.1	569.8	669.5	2,578.0
79	polybench/LA/kernels/mvt.bril	378.7	800.4	320.9	462.0	1,641.4
80	polybench/LA/solvers/cholesky.bril	577.2	924.3	584.5	620.1	2,228.0
81	polybench/LA/solvers/durbin.bril	247.1	516.4	198.9	324.5	1,094.4
82	polybench/LA/solvers/gramschmidt.bril	646.6	1,490.0	566.7	709.2	2,846.1
83	polybench/LA/solvers/lu.bril	528.7	807.0	498.2	591.9	1,982.8
84	polybench/LA/solvers/ludcmp.bril	939.0	1,536.1	901.6	997.0	3,532.7
85	polybench/LA/solvers/trisolv.bril	191.4	405.2	204.6	256.8	939.0
86	polybench/medley/deriche.rs	2,305.3	4,850.6	2,105.9	2,456.3	9,434.4
87	polybench/medley/floyd-warshall.rs	247.3	674.4	440.1	424.3	1,495.2
88	polybench/medley/nussinov.rs	423.8	835.8	670.7	539.5	2,065.7
89	polybench/stencils/adi.rs	790.0	2,727.3	738.9	1,088.9	4,405.6
90	polybench/stencils/fdtd-2d.rs	1,255.3	3,846.2	1,060.5	1,788.9	6,318.7
91	polybench/stencils/heat-3d.rs	1,236.2	4,002.9	1,097.1	1,338.9	6,487.7
92	polybench/stencils/jacobi-1d.rs	153.3	403.6	134.3	331.2	817.3
93	polybench/stencils/jacobi-2d.rs	471.9	1,608.0	411.5	723.0	2,635.1
94	polybench/stencils/seidel-2d.rs	326.3	1,090.3	313.6	471.0	1,860.8
95	raytrace/raytrace.rs	4,745.7	12,942.0	24,437.2	6,911.5	42,415.3

MAINTAINER BRIEF / 中文摘要

建议先在 egglog 的动作缓冲区前接入编译计划

eggcc 的真实生产链和独立重放给出了两个不同的答案。附录 B 记录原始 eggcc 程序如何在固定版本的生产引擎中执行、再由 Tiger 把结果传给下一 pass。附录 A 在同一个已经展开的输入上测试两个执行器。原生 egglog 直接执行未加 marker 的程序；Cell 由编译器生成 marker、共享入口、寄存器连接计划和跨 Run 的指令块。这个设计可以把推理收益同上一个 pass 选出了什么程序分开。

覆盖全部输入后，收益主要集中在哪里？ 共 96 个 passing 任务、288 个 pass，双方全部完成。三段推理相加，原生 egglog 共 143.43 秒，Cell 共 126.80 秒；73 个任务更快，23 个更慢。等权几何平均为 1.138×，按总耗时为 1.131×。Raytrace 占总节省时间的 82.6%。去掉它，按总时间仍有 1.029×。算法必须连同失利的例子一起看。3mm 的三段合计从 5.40 秒升至 6.02 秒。当前离线准备累计约 792 秒。若每个新程序都承担完整准备成本，只运行一次会慢于原生路径。

在哪个接口开始改最容易验证？ 先让编译器把同一输入变化编码为一份带槽位、事实引用和 epoch 的只读数据包，按规则原来的 Run 交给消费者。让已证明的完整键读取走寄存器计划，让剩余入口继续走原生 Free Join。两条路径都向 egglog 原有的动作缓冲区交付完整、有序的 RHS 输入。这样可以在保留原生 merge、rebuild 和 extraction 的条件下，直接比较每条绑定。

第二步才引入跨 Run 的事实引用。生产者已经构造过的结果，必须在下一消费者真正被调度时，验证其可见性、当前等价类和完整键。通过检查就切换到复用指令块，缺少任何必需端口就走普通路径。图 4 给出了实际的 SubTuple、Get、Single 链。

现有证据支持多强的结论？ sum-bits 两端投影到原始表后都是 3,206 行，但正文给出的具体 TermSingle 在 Cell 端可见，原生 egglog 的查询失败。相同 check 在原生执行 marked 源码时也失败。要声称可以替换后端，还需要定位首次语义分歧，核对可见事实和等价类，并用各自的 Tiger 结果独立驱动完整三 pass，对比提取程序及其行为。本报告支持一次可以逐步验收的上游架构实验。

6 NOVEMBER UPDATE / 内存与跨 **Run** 生命周期

让消费关系决定状态保留多久

本轮从 `27f098a2` 出发，在固定的 sum-bits 与 Raytrace Pass 3 上测试独立 Cell CPU 引擎。Raytrace 峰值 RSS 从 1,856.79 降到 1,432.68 MiB，减少 22.8%。原生 egglog 在未加 marker 的对应源码上，两次单线程测得 462.375 和 450.047 MiB，均包含解析、初始化和推理，未导出图。Cell 的峰值在导出前采样，包含编译产物加载。两类表示的内存比约从 4.07 倍降到 3.14 倍。

编译器首先列出共享入口的全部消费者。所有消费者都越过某个 epoch 后，引擎释放该 epoch 的旧材料化行，并保留准确的成员基数。后来的变更可以重建行；请求更早的 watermark 也会触发重建。Raytrace 留存的入口包从 665,584 降到 82,355。

另一项分析检查 Unit 完成表是否存在声明的读者。无人读取来源的完成记录继续保留键、值和修复身份，同时解除对生产者 Cell 的保活。完整 Cell 与紧凑来源快照合计从 687,913 降到 648,737。第三项显式选项在运行结构初始化后释放验证用 JSON 和输入缓冲；后续离线 Bake 需要重载原文件。

合用三项改动后，Raytrace 推理中位数为 6.746 → 6.717 秒，但逐对差异的中位数为 +16.663 ms，四对中只有一对更快。因此本轮证据主要支持内存收益。逐 Run 的 RHS、有效写入、Spine 接续、调度和最终图与冻结 Cell 参考一致。第 7 节的跨引擎具体语义差异仍需要单独解决。

分配统计还显示，History 产生的存活申请量约 2.3 MiB。更大的空间在图存储、参数、索引和共享入口里。最终图中生成的 View 与 Applied 表共占约 32.4% 的行；进一步压缩这些记录，需要证明它们在后续查询、union 转发、锚点发布与导出中的可观察行为。